UniDocs - Notes X Change

Praveen Ramisetti

Swetha Priya

Nandini

Mokshith

Sridhar

INTRODUCTION

UniDocs - Notes X Change is a full-stack university notes sharing web application designed to help students upload, browse, search, preview, vote on, and download academic study materials. The platform provides a centralized digital space where students can share notes across branches, semesters, and subjects, reducing the difficulty of finding reliable study resources.

The application is built using React for the frontend, Node.js and Express.js for the backend, and MongoDB for database storage. It includes secure user authentication using JWT, password hashing with bcryptjs, file upload support through Multer and Cloudinary, and a responsive user interface created with Tailwind CSS and Vite. UniDocs focuses on usability, speed, and organized access to academic content.

SCENARIO BASED CASE STUDY

Meet Rahul, a second-year engineering student who often struggles to collect proper notes before internal exams. His classmates store materials in different messaging groups, drives, and personal folders, making it difficult to find the correct file at the right time.

Rahul discovers UniDocs, a notes sharing platform created for university students. He registers an account, logs in securely, and starts browsing notes by semester, branch, subject, and subject code. When he finds useful notes, he can preview the file directly in the browser and download it for study.

Later, Rahul uploads his own Data Structures notes so other students can benefit from them. Other users upvote the notes if they are useful or downvote them if they are not relevant. This voting system helps better materials appear more prominently in the notes list.

User Registration and Authentication

Rahul creates an account using his name, email, and password. The password is securely hashed before being stored in the database. After login, Rahul receives a JWT token that allows him to access protected features such as uploading notes, managing his profile, and voting.

Notes Browsing and Searching

Rahul can browse all uploaded notes and filter them using semester, branch, subject, or subject code. This makes it easier to locate the exact academic material needed for a course.

Upload and Preview

Rahul uploads PDF or image-based notes with details such as title, subject, subject code, semester, and branch. Uploaded files are stored using Cloudinary, while metadata is saved in MongoDB.

Voting and Quality Control

Students can upvote or downvote notes. Each user can vote only once per note, and the system supports switching or removing votes. This helps the community identify useful study material.

Rahul's Experience

With UniDocs, Rahul no longer needs to search through many unrelated groups or folders. He can quickly find notes, contribute his own materials, and rely on community voting to identify high-quality academic resources.

SYSTEM REQUIREMENTS

Software Requirements

The following software tools are required for development, deployment, and usage of UniDocs:

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js: Version 18 or above recommended.
- npm: Required for installing frontend and backend dependencies.
- React.js: Used for building the user interface.
- Vite: Used as the frontend development and build tool.
- Node.js and Express.js: Used for creating backend REST APIs.
- MongoDB: Used as the NoSQL database for storing users, notes, and votes.
- Mongoose: Used to define schemas and interact with MongoDB.
- Cloudinary: Used for storing uploaded note files and profile images.
- Postman: Useful for API testing during development.
- Visual Studio Code: Preferred editor for development.
- Browser: Latest version of Google Chrome, Firefox, or Microsoft Edge.

Hardware Requirements

- Processor: Intel Core i5 or AMD Ryzen 5 and above.
- RAM: Minimum 8 GB, 16 GB recommended for smooth development.

- Storage: At least 1 GB free space for project files and dependencies.
- Display: 1366 x 768 or higher resolution.
- Internet: Required for MongoDB Atlas, Cloudinary, and package installation.

PROJECT ARCHITECTURE

UniDocs follows a client-server architecture. The frontend runs as a React single-page application, while the backend exposes REST APIs through Express.js. MongoDB stores structured application data, and Cloudinary stores uploaded files.

```
User
  |
  v
React Frontend - Vite
  |
  | HTTP requests using Axios
  v
Express Backend API
  |
  +--> MongoDB Database
  |
  +--> Cloudinary File Storage
```

TECHNICAL ARCHITECTURE

The application flow begins when a user interacts with the React frontend. Public users can view public pages, register, and log in. Authenticated users can upload notes, view note details, update their profile, and vote on notes.

The frontend sends requests to the Express backend. The backend verifies input, validates JWT tokens for protected routes, processes business logic in controllers, and communicates with MongoDB through Mongoose models.

When files are uploaded, Multer handles multipart form data and Cloudinary stores the file. MongoDB stores only the file URL and related note metadata. This keeps the database lightweight and allows uploaded files to be served from a cloud storage provider.

ER DIAGRAM

The UniDocs database contains three major entities: User, Note, and Vote.

```
User
 - _id
 - name
 - email
 - password
 - bio
 - college
 - profilePhoto
```

```
 - createdAt

Note
 - _id
 - title
 - subject
 - subjectCode
 - semester
 - branch
 - fileUrl
 - fileType
 - uploadedBy
 - upvotes
 - downvotes
 - createdAt

Vote
 - _id
 - userId
 - noteId
 - voteType
```

Relationships

- One user can upload many notes.
- Each note belongs to one uploaded user.
- One user can vote on many notes.
- One note can have many votes.
- The Vote model connects User and Note using userId and noteId.
- A compound unique index on userId and noteId prevents duplicate votes by the same user on the same note.

KEY FEATURES

- User registration and login with JWT authentication.
- Password hashing using bcryptjs.
- Protected routes for upload, profile, voting, and dashboard pages.
- Upload PDF or image notes.
- Store files in Cloudinary.
- Browse notes with pagination.
- Filter notes by semester, branch, subject, and subject code.
- Preview PDF and image files.
- Upvote and downvote notes.
- Toggle or switch existing votes.
- Owner-only note deletion.
- Profile update with name, bio, college, and profile photo.

- Responsive frontend interface.
- Toast notifications for user actions.
- Error boundary and not found page for better reliability.

ROLES AND RESPONSIBILITIES

Student/User

- Register and maintain an account.
- Login securely using valid credentials.
- Upload academic notes with correct subject details.
- Browse and search notes uploaded by other students.
- Preview and download study materials.
- Vote on notes based on usefulness.
- Manage personal profile information.
- Delete only the notes uploaded by their own account.

System/Admin Responsibility

The current codebase does not contain a separate admin panel. However, the system is responsible for:

- Authenticating users using JWT.
- Protecting private API routes.
- Validating note upload data.
- Managing file upload and deletion through Cloudinary.
- Preventing duplicate votes by the same user.
- Returning structured API responses.
- Handling errors gracefully.

USER FLOW

Student Flow

1. Open the UniDocs application.
2. Register a new account or login with existing credentials.
3. Browse available notes from the home/dashboard page.
4. Apply filters such as semester, branch, subject, or subject code.
5. Open a note detail page to preview the uploaded file.
6. Upvote or downvote the note.
7. Upload new notes through the upload page.
8. Manage personal information on the profile page.
9. View personal uploads and voted notes through dashboard-related features.

Upload Flow

10. User opens the upload page.
11. User enters title, subject, subject code, semester, and branch.
12. User selects a PDF or image file.
13. Frontend sends multipart form data to the backend.
14. Backend verifies JWT token.
15. File is uploaded to Cloudinary.
16. Note metadata and file URL are saved in MongoDB.
17. Newly uploaded note becomes available for other students.

MVC PATTERN

UniDocs follows the Model-View-Controller style on the backend.

Model Layer

The Model layer defines the data structure using Mongoose schemas.

- User.js stores account and profile information.
- Note.js stores note metadata, file URL, branch, semester, and vote counts.
- Vote.js stores user-note vote relationships.

Controller Layer

The Controller layer contains business logic.

- authController.js handles registration and login.
- noteController.js handles note upload, listing, detail view, user uploads, and deletion.
- voteController.js handles upvote, downvote, vote toggle, and user vote history.
- profileController.js handles profile fetching, profile update, and profile photo update.

View/Route Layer

In this REST API project, routes act as the view-facing layer for HTTP communication.

- authRoutes.js defines authentication endpoints.
- noteRoutes.js defines note and voting endpoints.
- profileRoutes.js defines profile endpoints.

Advantages of MVC in This Project

- Clear separation of database, logic, and route responsibilities.
- Easier maintenance and debugging.
- New features can be added by creating new routes, controllers, and models.
- Controllers can be tested independently.
- Multiple developers can work on separate layers with fewer conflicts.

PROJECT SETUP AND CONFIGURATION

Creating Project Folder

18. Create a main project folder named UniDocs.
19. Inside the folder, create two folders: client and server.
20. Open the main folder in Visual Studio Code.

Client Setup

```
cd client
npm install
npm run dev
```

Create a client environment file:

```
VITE_API_URL=http://localhost:5000
```

Server Setup

```
cd server
npm install
npm run dev
```

Create a server environment file:

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/unidocs
JWT_SECRET=your_super_secret_key
NODE_ENV=development
CLIENT_URL=http://localhost:5173
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

BACKEND DEVELOPMENT

Backend Structure

```
server/
  config/
    cloudinary.js
    db.js
  controllers/
    authController.js
    noteController.js
    profileController.js
    voteController.js
  middlewares/
    authMiddleware.js
    upload.js
  models/
    Note.js
    User.js
```

```
    Vote.js
  routes/
    authRoutes.js
    noteRoutes.js
    profileRoutes.js
  server.js
```

Backend Modules

config

- db.js connects the backend to MongoDB using Mongoose.
- cloudinary.js configures Cloudinary for uploaded files and profile photos.

controllers

- authController.js: Handles user registration, login, password hashing, and token generation.
- noteController.js: Handles note upload, search, filtering, retrieval, user notes, and deletion.
- voteController.js: Handles upvote, downvote, vote switching, and vote history.
- profileController.js: Handles profile retrieval, update, and profile photo upload.

middlewares

- authMiddleware.js: Verifies JWT tokens and protects private routes.
- upload.js: Handles file upload processing through Multer and Cloudinary storage.

routes

- authRoutes.js: Provides authentication routes.
- noteRoutes.js: Provides note and vote routes.
- profileRoutes.js: Provides profile routes.

API Endpoints

Authentication

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Register a new user	No
POST	/api/auth/login	Login user and return JWT	No

Notes

Method	Endpoint	Description	Auth Required
GET	/api/notes	List notes with filters and pagination	No
GET	/api/notes/:id	Get note details	No
POST	/api/notes/upload	Upload a new note	Yes
GET	/api/notes/my-uploads	Get notes uploaded by current user	Yes
DELETE	/api/notes/:id	Delete own note	Yes

Voting

Method	Endpoint	Description	Auth Required
POST	/api/notes/:id/vote	Upvote, downvote, switch, or remove vote	Yes

GET	/api/notes/:id/myvote	Get current user's vote on a note	Yes
GET	/api/notes/my-votes	Get notes voted by current user	Yes

Profile

Method	Endpoint	Description	Auth Required
GET	/api/profile	Get current user profile	Yes
PUT	/api/profile	Update profile details	Yes
PUT	/api/profile/photo	Update profile photo	Yes

DATABASE DEVELOPMENT

Configure MongoDB

MongoDB is used as the database for UniDocs. Mongoose is used to connect with MongoDB and define schemas.

Database Connection

The backend calls connectDB() from server/config/db.js when the server starts. This function reads the MongoDB connection string from the MONGO_URI environment variable and connects the Express application to the database.

Schemas

User.js

The User schema stores student account information:

- name
- email
- password
- bio
- college
- profilePhoto
- createdAt

Note.js

The Note schema stores uploaded note information:

- title
- subject
- subjectCode
- semester
- branch
- fileUrl
- fileType

- uploadedBy
- upvotes
- downvotes
- createdAt

Vote.js

The Vote schema stores each user's voting action on a note:

- userId
- noteId
- voteType

The schema also defines a unique compound index on userId and noteId so a user cannot create multiple votes for the same note.

FRONT-END DEVELOPMENT

Frontend Structure

```
client/
  src/
    assets/
    components/
      AuthRoute.jsx
      CampusWatermark.jsx
      ErrorBoundary.jsx
      Layout.jsx
      NoteCard.jsx
      ProtectedRoute.jsx
      PublicLayout.jsx
      WaterBackground.jsx
    context/
      AuthContext.jsx
    pages/
      AboutPage.jsx
      ContactPage.jsx
      Dashboard.jsx
      FeaturesPage.jsx
      Home.jsx
      LandingPage.jsx
      Login.jsx
      NoteDetail.jsx
      NotFound.jsx
      Profile.jsx
      PublicHome.jsx
      Register.jsx
      Upload.jsx
    App.jsx
    main.jsx
```

Pages

- PublicHome.jsx: Public landing/home experience.
- AboutPage.jsx: Information about the platform.
- FeaturesPage.jsx: Highlights major features.
- ContactPage.jsx: Contact page.
- Login.jsx: User login screen.
- Register.jsx: User registration screen.
- Home.jsx: Main authenticated notes browsing page.
- Upload.jsx: Note upload page.
- NoteDetail.jsx: Detailed note preview page.
- Profile.jsx: User profile management page.
- Dashboard.jsx: User activity/dashboard page.
- NotFound.jsx: Page shown for invalid routes.

Components

- Layout.jsx: Main authenticated layout wrapper.
- PublicLayout.jsx: Layout for public pages.
- ProtectedRoute.jsx: Restricts access to authenticated users.
- AuthRoute.jsx: Handles route behavior for authenticated and unauthenticated users.
- NoteCard.jsx: Displays note summary information.
- ErrorBoundary.jsx: Handles frontend runtime errors gracefully.
- WaterBackground.jsx and CampusWatermark.jsx: Visual background components.

Frontend Routing

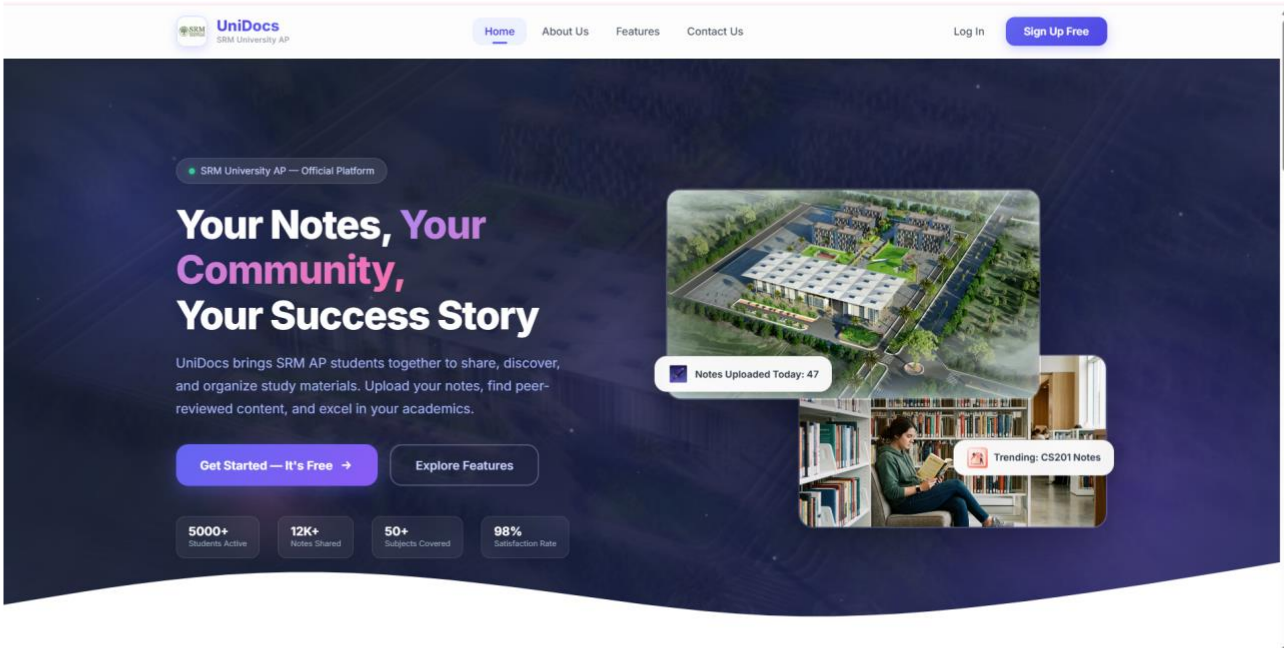
React Router is used for navigation. Public pages are available before login, while upload, note detail, profile, and dashboard routes are protected.

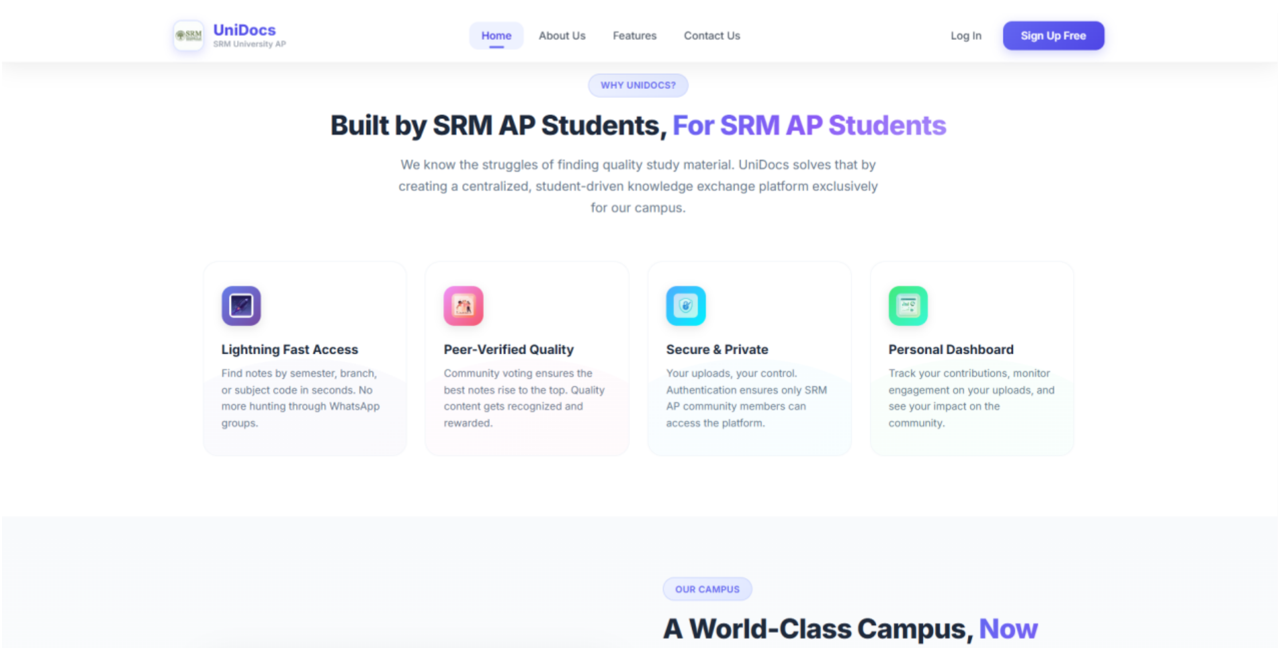
Frontend Libraries

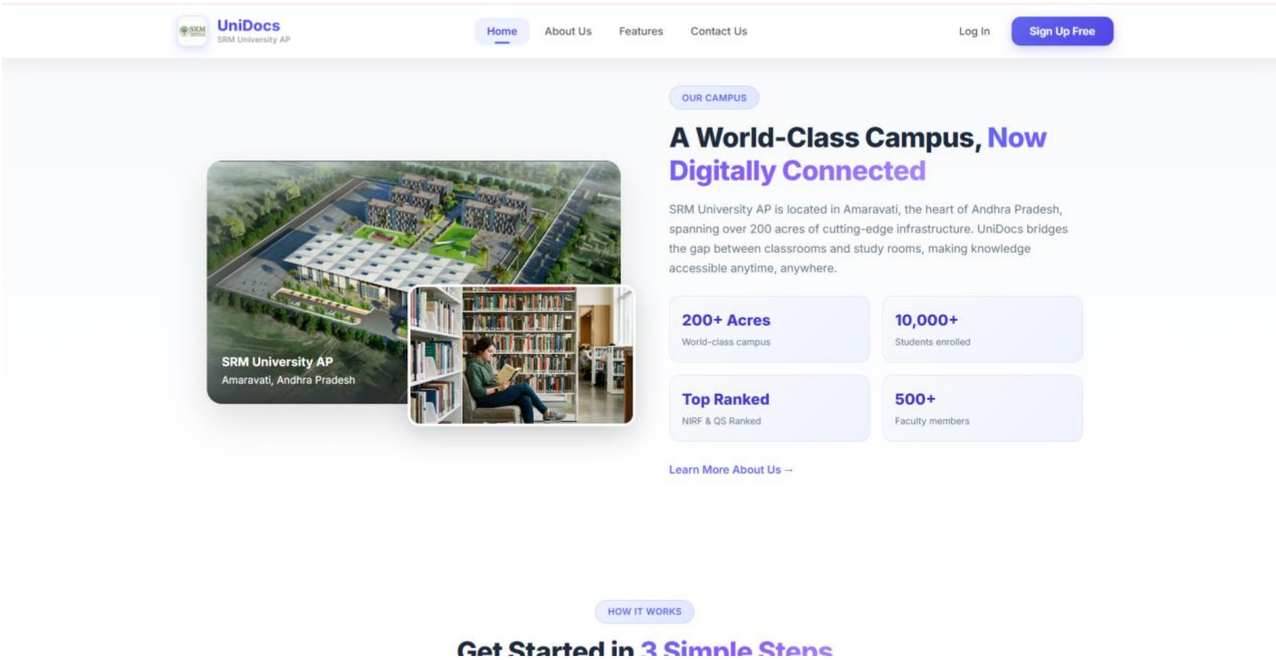
- React for UI development.
- React Router DOM for routing.
- Axios for API requests.
- React Hot Toast for notifications.
- Tailwind CSS for responsive styling.
- Vite for development and production build.

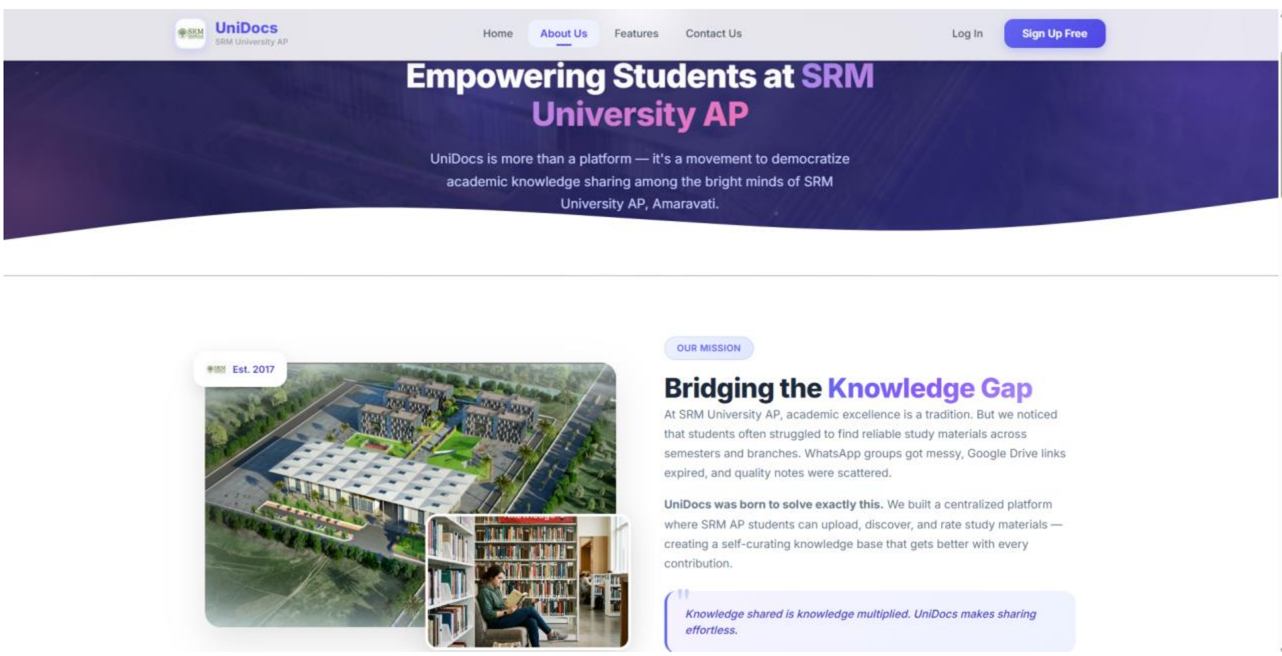
OUTPUT SCREENSHOTS

Landing Page






UniDocs
SRM University AP
Home
About Us
Features
Contact Us
Log In
Sign Up Free
OUR CAMPUS
A World-Class Campus, Now Digitally Connected
SRM University AP is located in Amaravati, the heart of Andhra Pradesh, spanning over 200 acres of cutting-edge infrastructure. UniDocs bridges the gap between classrooms and study rooms, making knowledge accessible anytime, anywhere.
SRM University AP
Amaravati, Andhra Pradesh
200+ Acres
World-class campus
10,000+
Students enrolled
Top Ranked
NIRF & QS Ranked
500+
Faculty members
Learn More About Us →
HOW IT WORKS
Get Started in 3 Simple Steps


UniDocs
SRM University AP
Home
About Us
Features
Contact Us
Log In
Sign Up Free
Empowering Students at SRM University AP
UniDocs is more than a platform — it's a movement to democratize academic knowledge sharing among the bright minds of SRM University AP, Amaravati.
Est. 2017
OUR MISSION
Bridging the Knowledge Gap
At SRM University AP, academic excellence is a tradition. But we noticed that students often struggled to find reliable study materials across semesters and branches. WhatsApp groups got messy, Google Drive links expired, and quality notes were scattered.
UniDocs was born to solve exactly this. We built a centralized platform where SRM AP students can upload, discover, and rate study materials — creating a self-curating knowledge base that gets better with every contribution.
Knowledge shared is knowledge multiplied. UniDocs makes sharing effortless.

interface. Tag with semester, branch, and subject code for easy discovery by your peers.

- ✓ Drag & Drop Upload
- ✓ Auto-tagging
- ✓ Multiple Formats
- ✓ Cloud Storage

Advanced Search & Filter

Find exactly the notes you need in seconds. Filter by semester, branch, subject code, or popularity. Our smart search understands what you're looking for.

- ✓ Subject Code Search
- ✓ Semester Filter
- ✓ Branch Filter
- ✓ Popularity Sort

Community Voting System

Quality rises to the top organically. Upvote the best notes and downvote low-quality content. The community decides what's worth studying.

Visit Us	Email Us	Call Us	Online
SRM University AP Neerukonda, Mangalagiri Mandal Guntur District, Andhra Pradesh 522240	praveen_ramisetti@srmap.edu.in	+91 866 242 8000 Mon-Fri: 9:00 AM - 5:00 PM IST	srmap.edu.in

Send Us a Message

Fill out the form below — your message will be sent to **praveen_ramisetti@srmap.edu.in**

FULL NAME
Your full name

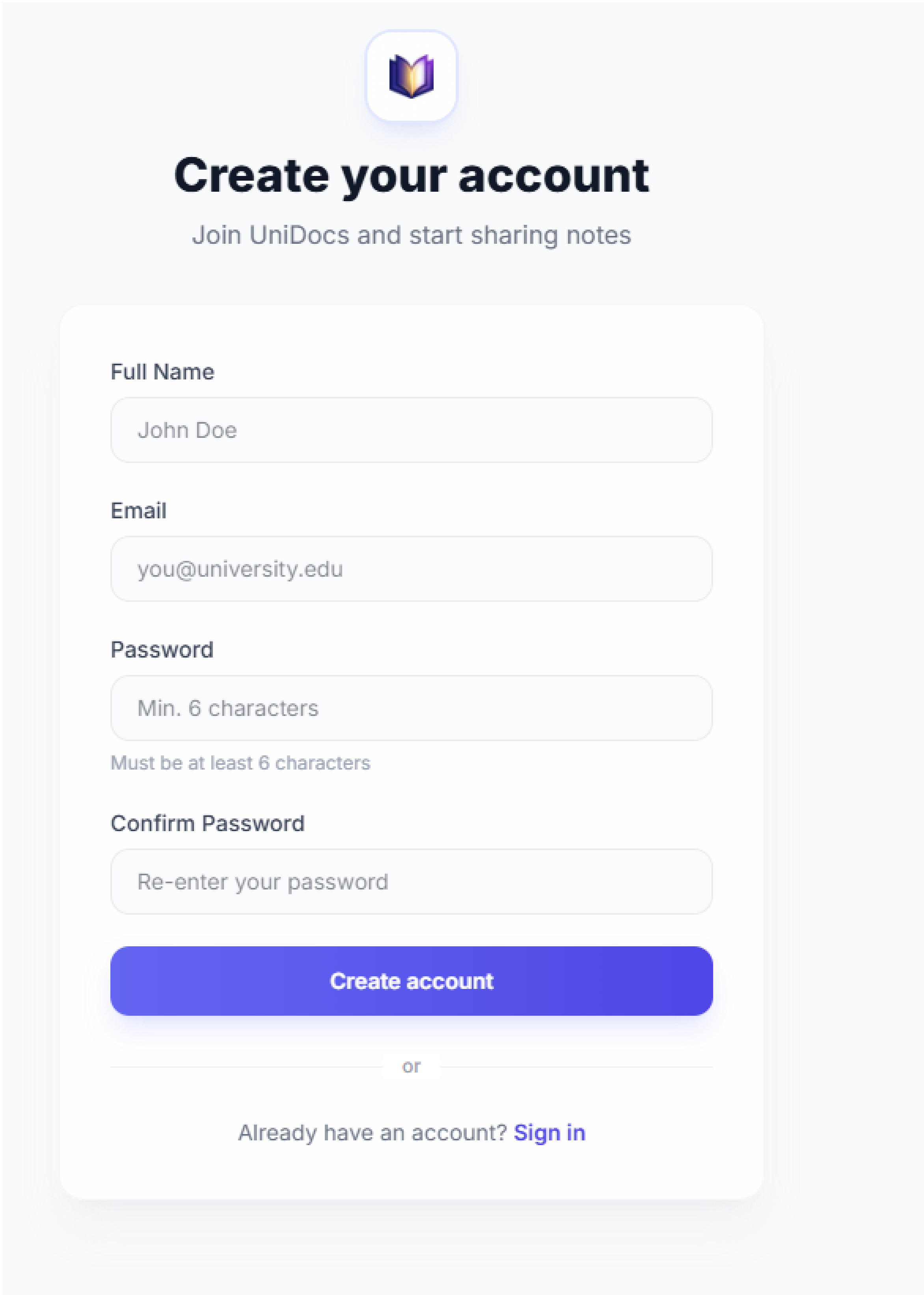
EMAIL
you@srmap.edu.in

SUBJECT
What's this about?

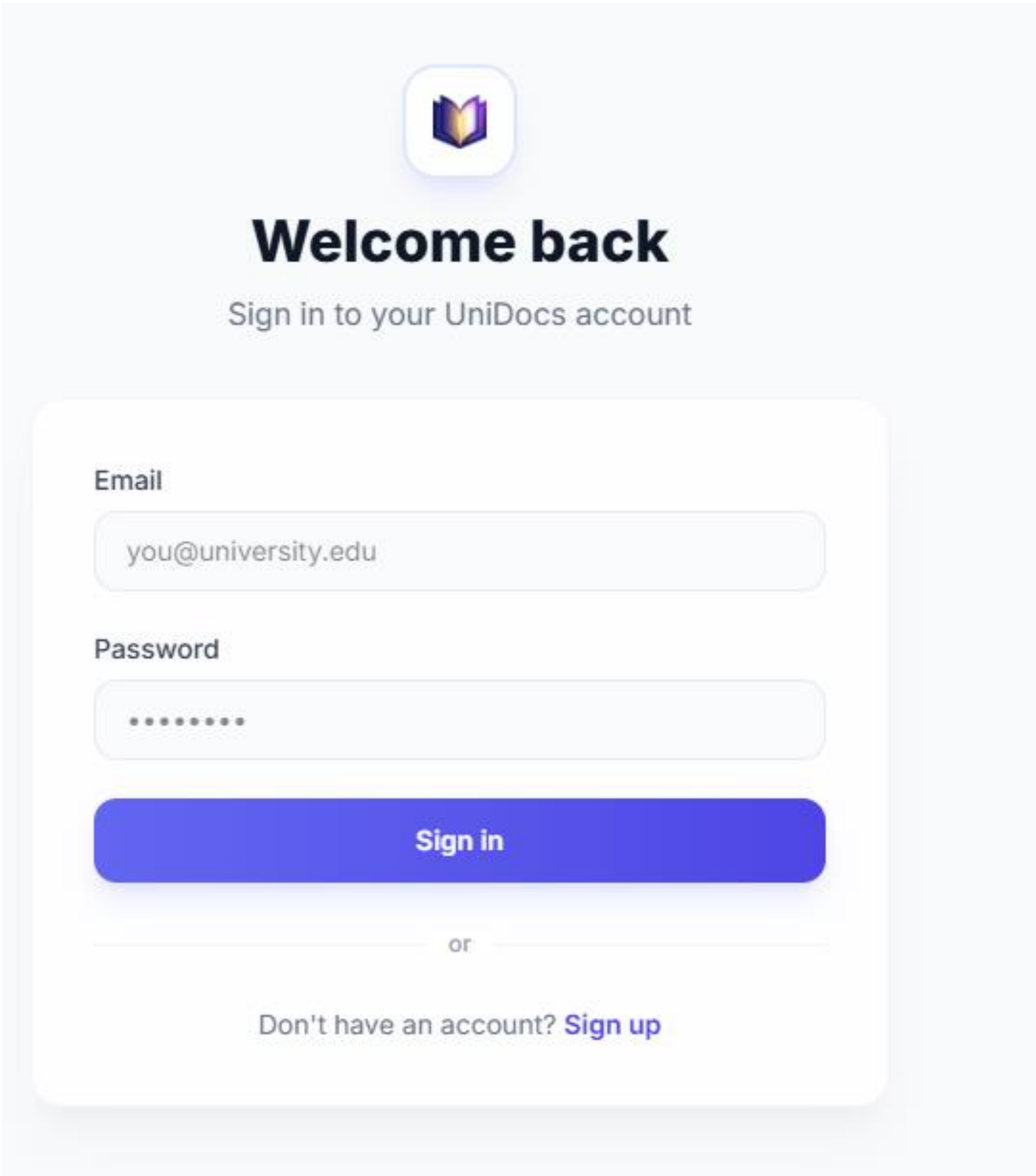
MESSAGE
Write your message here...

Send Message

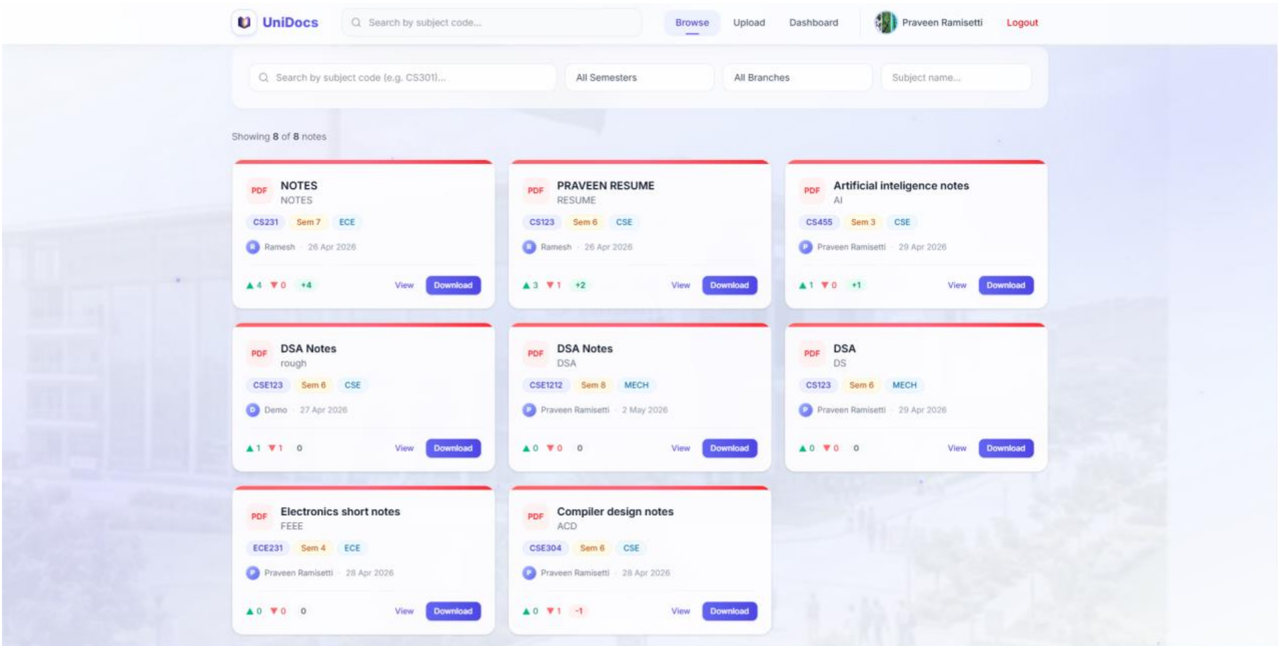
Register Page



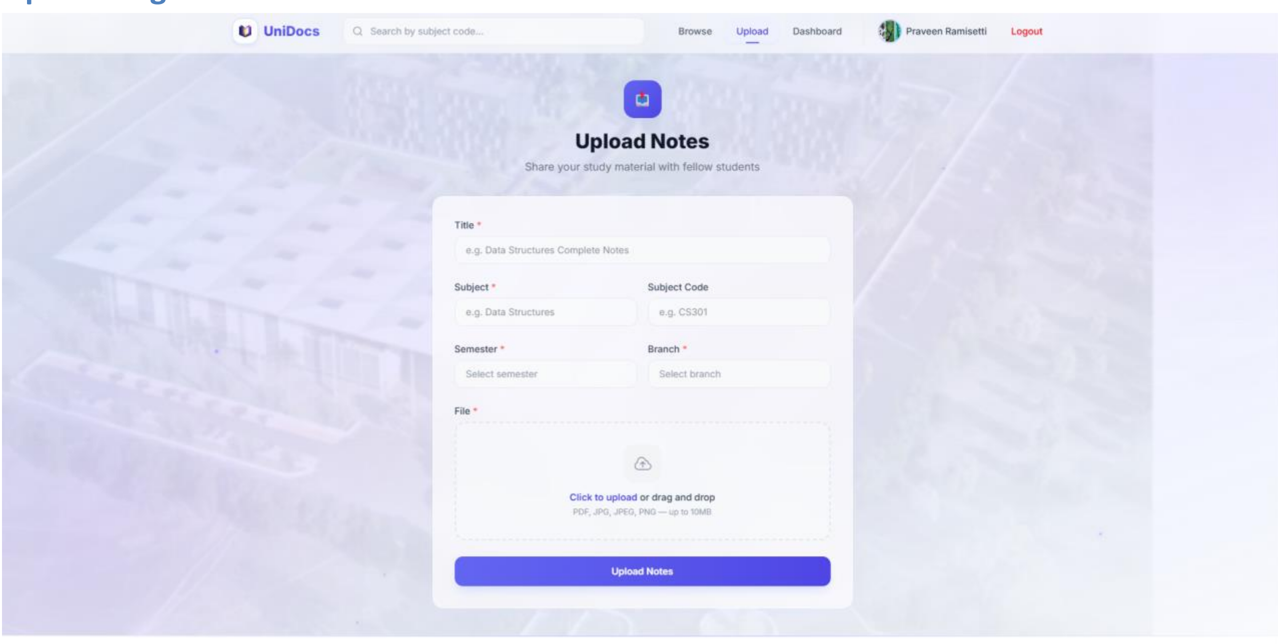
Login Page


Welcome back
Sign in to your UniDocs account
Email
you@university.edu
Password
Sign in
or
Don't have an account? Sign up

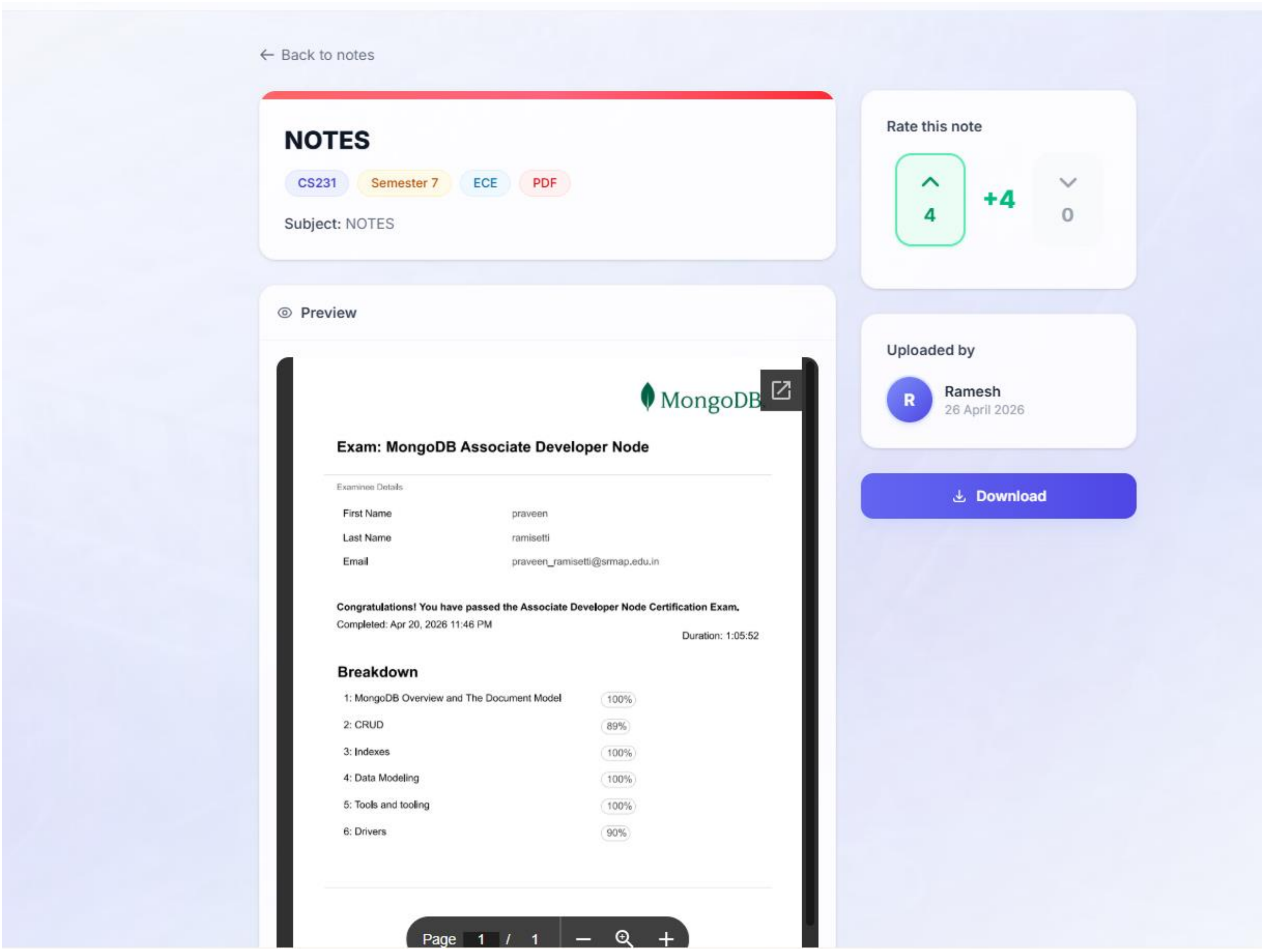
Home / Notes Listing Page



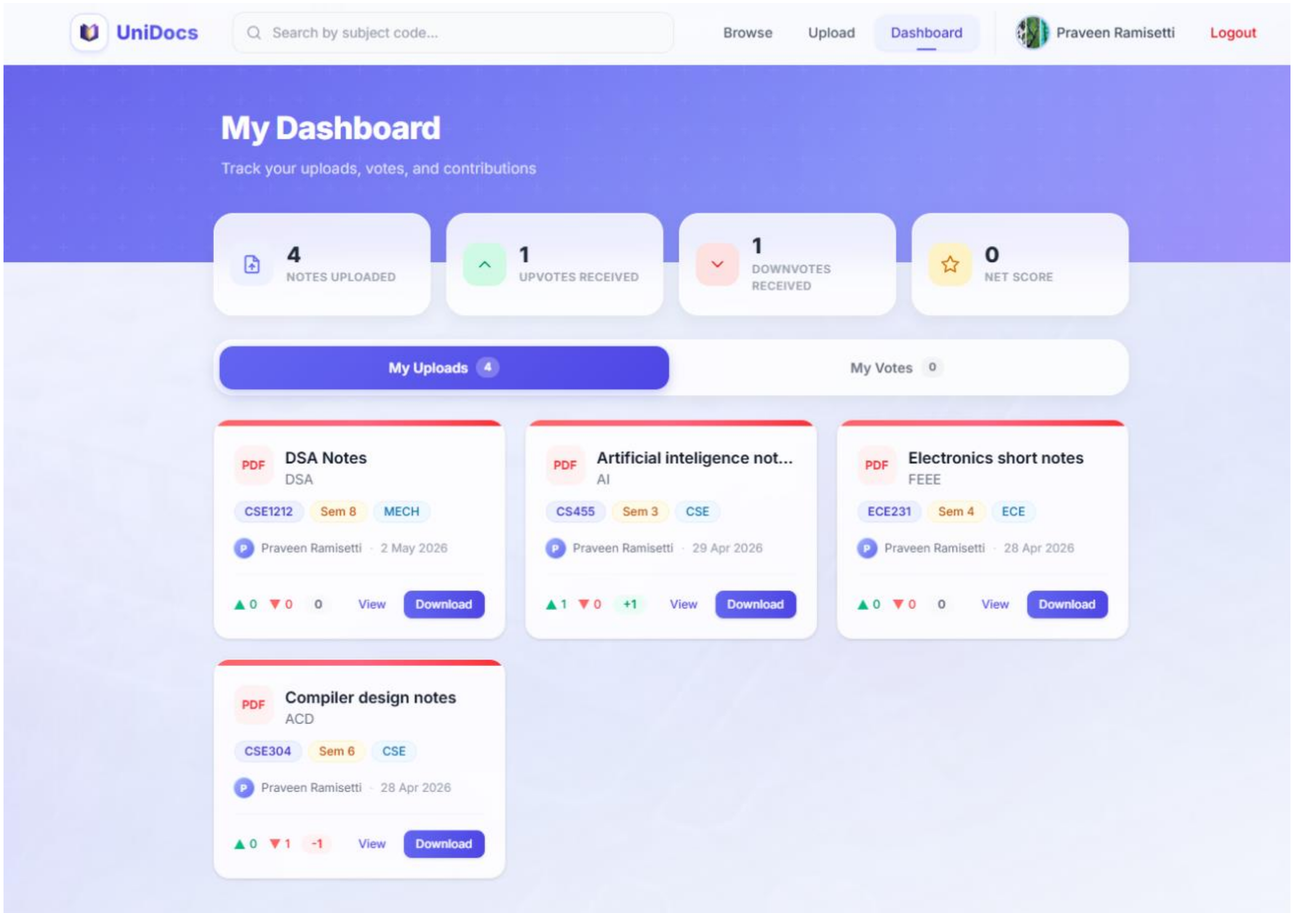
Upload Page



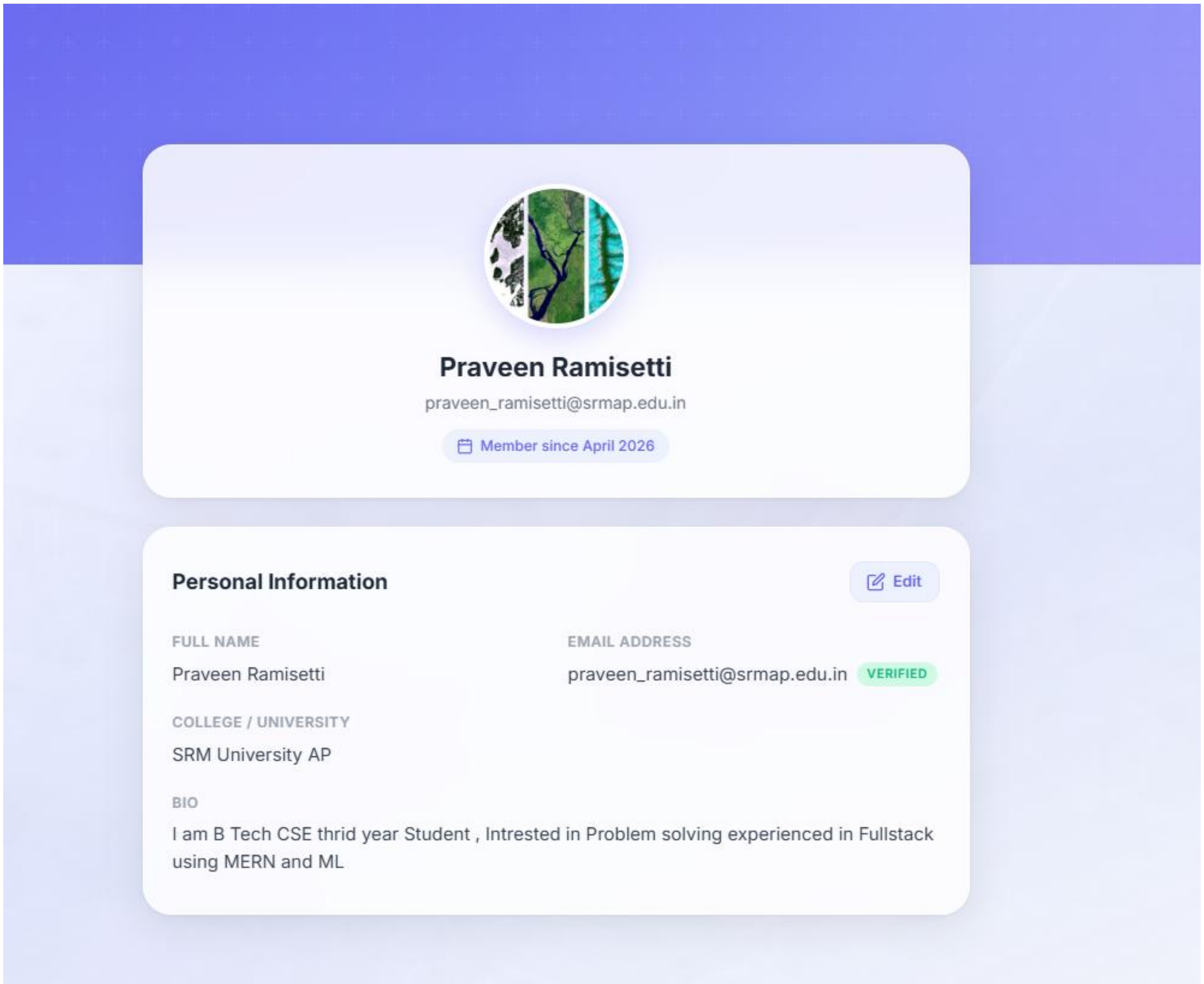
Note Detail Page



Profile Page



Dashboard Page



DEMO VIDEO LINK

https://drive.google.com/file/d/1O7-qHP-AdtykrJcJR5sLiN9798ZvHxaH/view

https://drive.google.com/file/d/1sUouxUdxTb5uq8DHrY-OzB30UKRam3-x/view?usp=sharing

CODE DRIVE / GITHUB LINK”

https://github.com/Praveenramisetti76/UniDocs_notesXchange

CONCLUSION

UniDocs - Notes X Change provides a practical solution for university students who need organized access to academic notes. By combining secure authentication, cloud-based file uploads, search and filtering, preview support, and community voting, the platform improves the way students share and discover study resources.

The project uses a modern full-stack architecture with React, Express.js, MongoDB, Mongoose, JWT, Multer, and Cloudinary. Its modular structure makes it easy to maintain and extend with future features such as admin moderation, comments, bookmarks, notifications, department-wise analytics, and advanced search.